

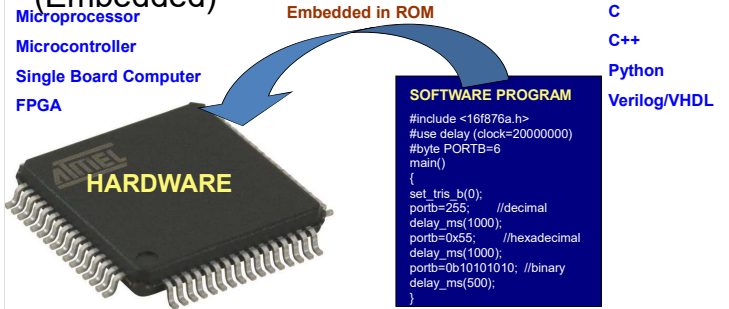
ELK346E
uP and uC
ARM
Basic Concepts
Computer and Assembly Language

Dr. Aydın Tarık Zengin
 Electrical Engineering
 with the courtesy of Dr. Ahmet Tekin and Dr. Enver Çavuş
 and embedded systems programming lessons by Quantum Leaps LLC

1

What is an Embedded System?

- Combination of Hardware & Software (Embedded)

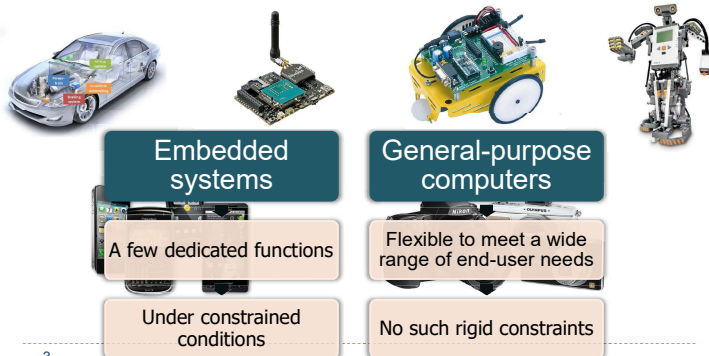


- Performs some functions or specific tasks

2

Embedded System?

An **embedded system** is a computer system designed to perform one or a few dedicated functions often with constraints, e.g., in real time, with low power or limited memory, ...



3

Why Do We Learn Embedded Systems?

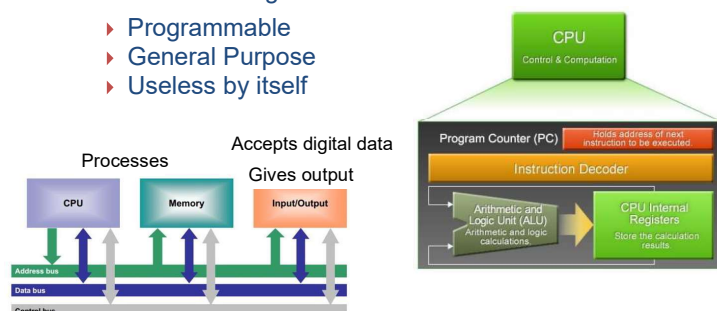
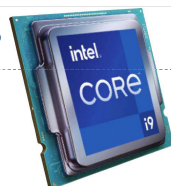
- They are everywhere



4

What is a Microprocessor (uP)?

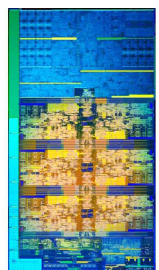
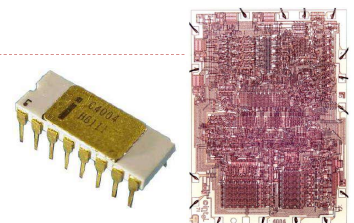
- A brain that controls hardware
- Also known as CPU
 - Central Processing Unit or Processor
 - Programmable
 - General Purpose
 - Useless by itself



5

Intel 4004 vs i9

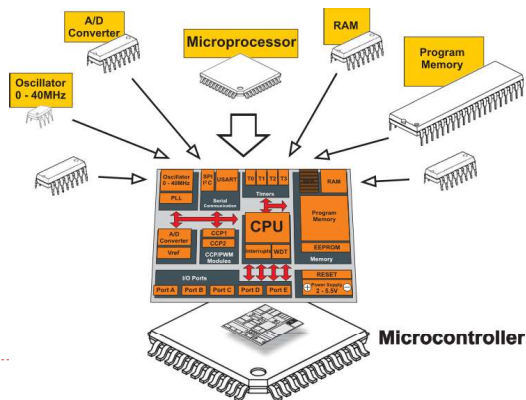
- First microprocessor (1971)
 - 10 μ m process
 - 2300 transistors
 - 400- 800 kHz
 - 4-bit word size
 - 16-pin DIP package
- i9 10th Gen (2020)
 - 14 nm process
 - 7 Billion transistors
 - 5.3 GHz



6

What is a Microcontroller (uC)?

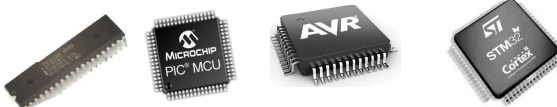
- ▶ All in One
 - ▶ Task specific



uP vs uC

Microprocessor	Micro Controller
Microprocessor is heart of Computer system.	Micro Controller is a heart of embedded system.
It is just a processor. Memory and I/O components have to be connected externally.	Micro controller has external processor along with internal memory and I/O components.
Since memory and I/O have to be connected externally, the circuit becomes large.	Since memory and I/O are present internally, the circuit is small.
Cannot be used in compact systems and hence inefficient.	Can be used in compact systems and hence it is an efficient technique.
Cost of the entire system increases.	Cost of the entire system is low.
Due to external components, the entire power consumption is high. Hence it is not suitable to be used with devices running on stored power like batteries.	Since external components are low, total power consumption is less and can be used with devices running on stored power like batteries.
Most of the microprocessors do not have power saving features.	Most of the micro controllers have power saving modes like idle mode and power saving mode. This helps to reduce power consumption even further.
Since memory and I/O components are all external, each instruction will need external operation, hence it is relatively slower.	Since components are internal, most of the operations are internal instruction, hence speed is fast.
Microprocessor have less number of registers, hence more operations are memory based.	Micro controller have more number of registers, hence the programs are easier to write.
Microprocessors are based on von Neumann model/architecture where program and data are stored in same memory module.	Micro controllers are based on Harvard architecture where program memory and Data memory are separate.
Mainly used in personal computers.	Used mainly in washing machine, MP3 players.

Different uCs



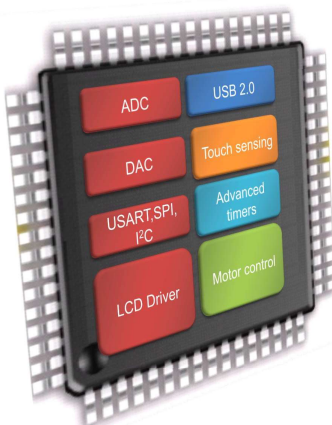
	8051	PIC	AVR	ARM
Bus width	8-bit for standard core	8/16/32-bit	8/32-bit	32/64-bit
Communication Protocols	UART, USART, SPI, I2C	PIC, UART, USART, LIN, CAN, Ethernet, SPI, I2S	UART, USART, SPI, I2C, (special purpose AVR CAN, USB, Ethernet)	UART, USART, LIN, I2C, SPI, CAN, USB, Ethernet, I2S, DSP, SAI (serial audio), IrDA
Speed	12 Clock/instruction cycle	4 Clock/instruction cycle	1 clock/ instruction cycle	1 clock/ instruction cycle
Memory	ROM, SRAM, FLASH	SRAM, FLASH	Flash, SRAM, EEPROM	Flash, SDRAM, EEPROM
ISA	CLSC	Some feature of RISC	RISC	RISC
Memory Architecture	Von Neumann	Harvard	Modified	Modified Harvard
Power Consumption	Average	Low	Low	Low
Families	8051 variants	PIC16, PIC17, PIC18, PIC24, PIC32	Tiny, Atmega, Xmega, special purpose AVR	ARMv4, 5, 6, 7 and series
Community	Vast	Very Good	Very Good	Vast
Manufacturer	NXP, Atmel, Silicon Labs, Dallas, Cypress, Infineon, etc.	Microchip	Atmel (now Microchip)	Apple, Nvidia, Qualcomm, Samsung, and TI etc.
Cost (as compared to features provide)	Very Low	Average	Average	Low
Other Feature	Known for its Standard	Cheap	Cheap, effective	High speed operation Vast
Popular Microcontrollers	AT89C51, P89v51, etc.	PIC18FXX8, PIC16F88X, PIC32MXX	Atmega8, 16, 32, Arduino Community	LPC2148, ARM Cortex-M0 to ARM Cortex-M7, etc.

Why ARM?

- ▶ A simple, efficient RISC core
 - ▶ Small die area
 - ▶ Low power
 - ▶ Low interrupt latency
 - ▶ Processing power with low current consumption
 - ▶ Good MIPS/Watt figure
 - ▶ Ideal for portable devices
- } Dominant in the cell phone market

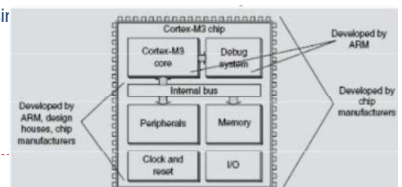
ARM Processor Market

- ▶ As of 2005, **98%** of the more than one billion mobile phones sold each year used ARM processors
- ▶ As of 2009, ARM processors accounted for approximately **90%** of all embedded 32-bit RISC processors
- ▶ In 2010 alone, **6.1 billion** ARM-based processor, representing of **95%** of digital televisions and set-top boxes and **10%** of mobile computers
- ▶ As of 2014, over 50 billion ARM processors have been produced



ARM Ltd

- ▶ Founded in 1990
 - ▶ Advanced RISC Machines
- ▶ Headquarters in Cambridge, UK
 - ▶ Design centers, support, and engineering offices all over the world
- ▶ Best known for its range of RISC processor cores designs
 - ▶ Other products – fabric IP, software tools, models, cell libraries - to help partners develop and ship ARM-based SoCs
- ▶ ARM does not manufacture silicon
 - ▶ Intellectual property (IP) licenser
 - ▶ RTL and synthesis flows
 - ▶ GDSII layout



[illegible]

Classic ARM Processors		Application Cortex Processors		Embedded Cortex Processors	
			Cortex-A15		
			Cortex-A9		
		ARM11MP	Cortex-A8		
	ARM926	ARM176JZ	Cortex-A7	SC300	SC000
SC100	ARM968	ARM1136J	Cortex-A5	Cortex-M3	Cortex-M1
ARM7TDMI	ARM946	ARM1156T2	Cortex-R4	Cortex-M4	Cortex-M0
ARMv4T	ARMv5TJ	ARMv6	ARMv7A/R	ARMv7M/ME	ARMv6M
ARM 32-Bit ISA					
Thumb 16-Bit ISA					
		Thumb-2 Mixed ISA		Thumb	Thumb
				Thumb-2	
VFPv2	VFPv2	VFPv3		NVIC	NVIC
Jazelle	Jazelle	Jazelle		WIC	WIC
	TrustZone	TrustZone			
	SIMD	SIMD			
		NEON			
		Virtualization			

The diagram illustrates the progression of ARM processors. The vertical axis represents 'Performance, Functionality' and the horizontal axis represents 'Capability'. Processors are categorized into 'Classic' (blue boxes) and 'Embedded' (green boxes).

Processor	Category	Relative Performance/Functionality	Relative Capability
ARM7TDMI-S	Classic	Low	Low
ARM946-S	Classic	Medium-Low	Low-Medium
ARM968-S	Classic	Medium	Medium
ARM1156T2(F)-S	Classic	High	Medium-High
Cortex-M0	Embedded	Low	Low
Cortex-M1	Embedded	Low-Medium	Low-Medium
Cortex-M3	Embedded	Medium-Low	Medium-Low
Cortex-M4	Embedded	Medium	Medium
Cortex-R4	Embedded	Medium-High	Medium-High
Cortex-R5	Embedded	High	High
Cortex-R7	Embedded	Very High	Very High

Performance, Functionality

Classic ARM Processors

Application Cortex Processors

ARM926EJ-S

ARM1136(F)-S

ARM1176JZ(F)-S

ARM11MP

Cortex-A5

Cortex-A7

Cortex-A8

Cortex-A9

Cortex-A15

Capability

v4	v5	v6	v7
Halfword and signed halfword / byte support	Improved interworking CLZ Saturated arithmetic DSP MAC instructions	SIMD Instructions Multi-processing v6 Memory architecture Unaligned data support	Thumb-2 Architecture Profiles 7-A -Applications 7-R -Real-time 7-M -Microcontroller
System mode		Extensions: Thumb-2 (6T2) TrustZone® (6Z) Multicore (6K)	
Thumb instruction set (v4T)	Extensions: Jazelle (5TEJ)	Thumb only (6-M)	

- **Note that implementations of the same architecture can be different**
 - Cortex-A8 - architecture v7-A, with a 13-stage pipeline
 - Cortex-A9 - architecture v7-A, with an 8-stage pipeline

Apple A14 Bionic
is a custom design
System-on-Chip (SoC)
based on a
64-bit ARMv8.5a

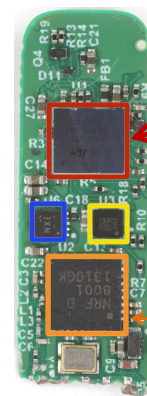
Apple Watch



- ▶ Apple S1 Processor
 - ▶ **32-bit ARMv7-A** compatible
 - ▶ # of Cores: **1**
 - ▶ CMOS Technology: 28 nm
 - ▶ L1 cache 32 KB data
 - ▶ L2 cache 256 KB
 - ▶ GPU PowerVR SGX543

19

Fitbit Flex Teardown



STMicroelectronics
32L151C6 Ultra Low
Power **ARM Cortex M3**
Microcontroller

Nordic Semiconductor
nRF8001 Bluetooth Low
Energy Connectivity IC

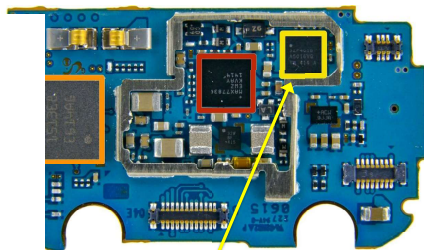
20

www.ifixit.com

Samsung Galaxy Gear



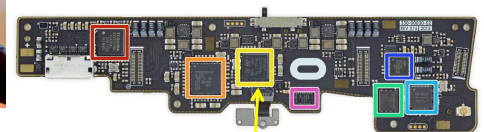
source: ifixit.com



- ▶ STMicroelectronics
STM32F401B **ARM-Cortex M4** MCU with
128KB Flash

21

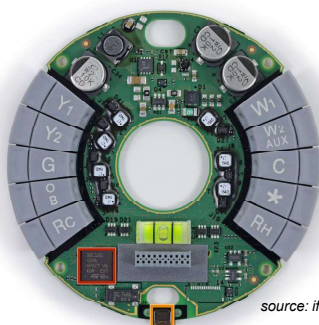
Oculus VR



- ▶ Facebook's \$2 Billion Acquisition Of Oculus in 2014 source: ifixit.com
- ▶ ST Microelectronics STM32F072VB **ARM Cortex-M0** 32-bit RISC Core Microcontroller

22

Nest Learning Thermostat

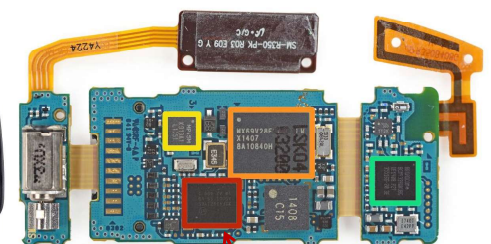


source: ifixit.com

- ▶ ST Microelectronics **STM32L151VB** ultra-low-power 32 MHz **ARM Cortex-M3** MCU

23

Samsung Gear Fit Fitness Tracker

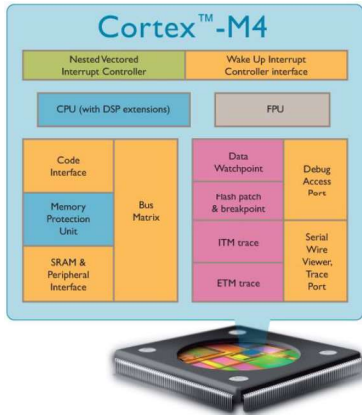


source: ifixit.com

- ▶ STMicroelectronics **STM32F439ZI** 180 MHz, 32 bit **ARM Cortex-M4** CPU

24

Our Class: Cortex-M4



- **ARMv7E-M Architecture**
 - Thumb-2 only
 - DSP extensions
- **Optional FPU (Cortex-M4F)**
- **Otherwise, same as Cortex-M3**
- **Implements full Thumb-2 instruction set**
 - Saturated math (e.g. QADD)
 - Packing and unpacking (e.g. UXTB)
 - Signed multiply (e.g. SMULTB)
 - SIMD (e.g. ADD8)

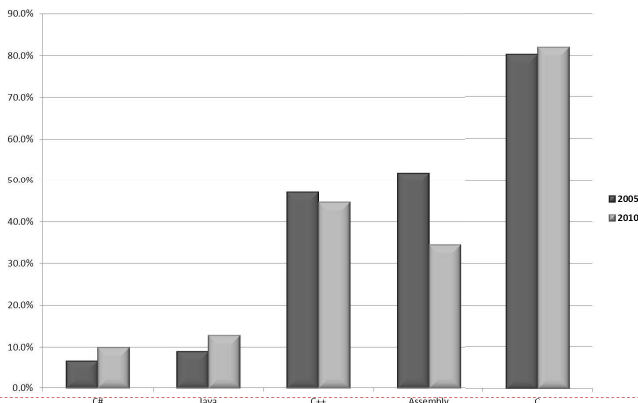
▶ 25

Why Use C?

- ▶ It is a 'mid-level', with 'high-level' and 'low-level' features
 - ▶ Support for functions and modules
 - ▶ Good access to hardware via pointers
- ▶ It is very efficient;
- ▶ It is popular and well understood;
- ▶ Even desktop developers who have used only Java or C++ can soon understand C syntax;
- ▶ Good, well-proven compilers are available for every embedded processor (8-bit to 32-bit or more);
- ▶ Experienced staff are available;
- ▶ Plenty of resources
 - ▶ Books, training courses, code samples and websites

26

Programming Language Use in Embedded



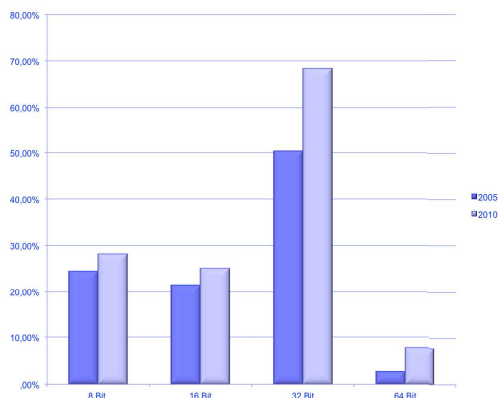
▶ 27

Why Do We Learn Assembly?

- ▶ Help you understand how does the processor work
- ▶ Assembly program runs faster than high-level language. Performance critical codes must be written in assembly.
 - ▶ Use the profiling tools to find the performance bottle and rewrite that code section in assembly
 - ▶ Latency-sensitive applications, such as aircraft controller
 - ▶ Standard C compilers do not use some operations available on ARM processors, such as ROR (Rotate Right) and RRX (Rotate Right Extended).
- ▶ Hardware/processor specific code,
 - ▶ Processor booting code
 - ▶ Device drivers
 - ▶ A test-and-set atomic assembly instruction can be used to implement locks and semaphores.
- ▶ Cost-sensitive applications
 - ▶ Embedded devices, where the size of code is limited, wash machine controller, automobile controllers
- ▶ The best applications are written by those who've mastered assembly language or fully understand the low-level implementation of the high-level language statements they're choosing.

28

Processor Use in Embedded Designs



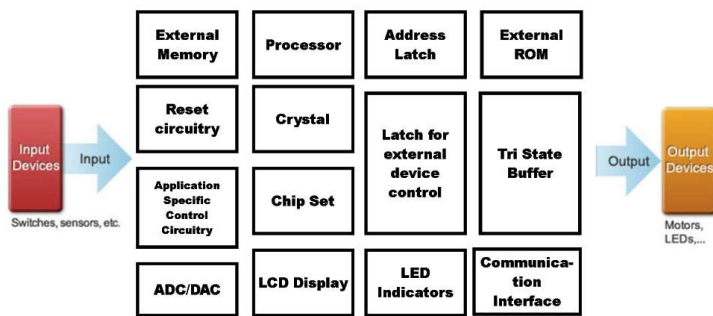
▶ 29

BASIC CONCEPTS

▶ 30

Basic Concepts

Functional Block diagram of the hardware architecture of an embedded system



31

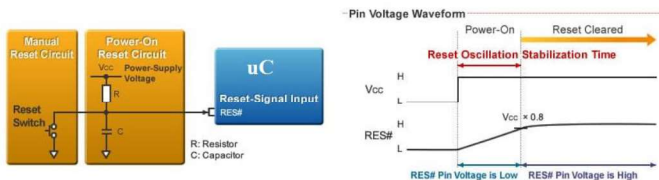
Reset Circuitry

- ▶ Built in the hardware to take care of any unforeseen problems.
 - ▶ Software hang-ups, power supply failures, etc.
- ▶ The processor sends a status signal to this circuit periodically.
- ▶ If in case this signal is not received, it is an indicator that something is wrong, which is then reset by this circuit.
- ▶ Power on Reset
 - ▶ After power-on, internal circuitry needs time to stabilize.
 - ▶ During this interval, the uP cannot be expected to perform normally.
 - ▶ Applying a reset signal to the reset pin

32

Power on Reset Circuitry

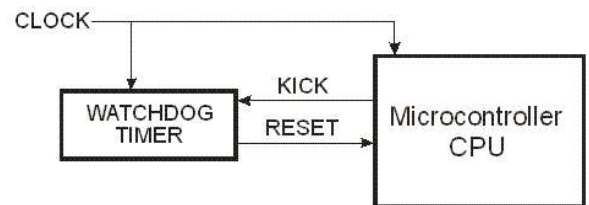
- ▶ The incoming power voltage moves through this circuit's resistor,
 - ▶ Some of the initial current to flow into the capacitor.
 - ▶ The voltage rise into the reset pin is gradual.
 - ▶ The reset condition is cleared when the rising voltage reaches a predetermined level.



33

Watchdog Timer (WDT): Software Reset

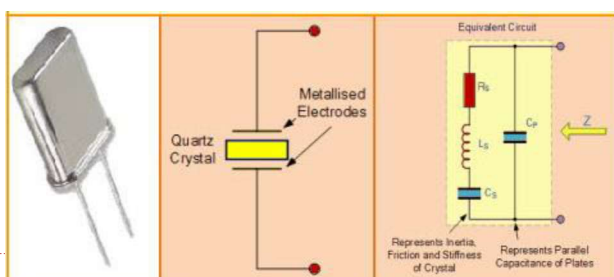
- ▶ Resets the processor in response to an invalid SW state.
- ▶ Uses a counter register
 - ▶ A clock source continuously increments the counter register,
 - ▶ for every executed instruction, this counter register's count is cleared.
 - ▶ If any instruction fails to execute, then the WDT counter not cleared
 - ▶ when it overflows, the watchdog logic forces a system reset.



34

Crystal Oscillator (Clock Generator)

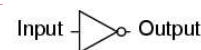
- ▶ Mechanical resonance of a vibrating crystal of piezoelectric material
 - ▶ an electrical signal with a very precise frequency
- ▶ The most common type: the quartz crystal,
 - ▶ became known as "crystal oscillators."



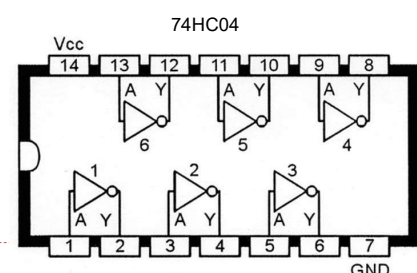
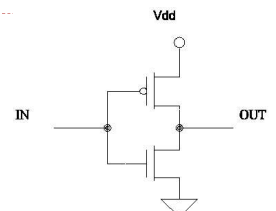
35

Inverters

- ▶ Not Gate



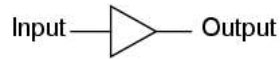
Input	Output
0	1
1	0



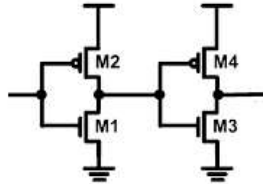
36

Buffers

- As if double inversion



Input	Output
0	0
1	1

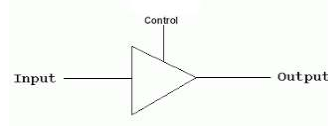


37

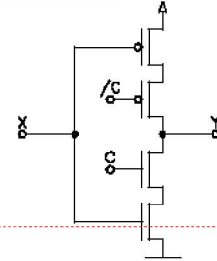
Tri-State Buffers

- Provides isolation between a shared input and output pin

74HC125 74HC244

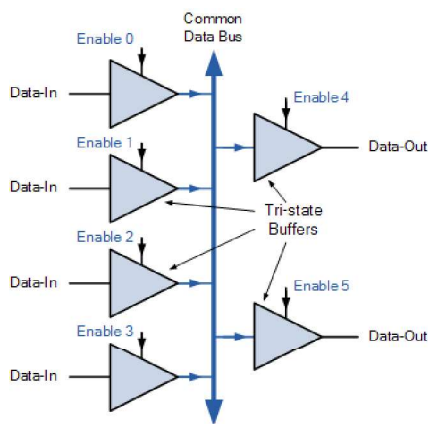


In	Control	Out
0	0	Hi Z
1	0	Hi Z
0	1	0
1	1	1



38

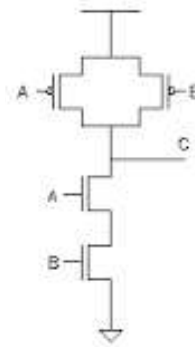
Tri-State Buffer Usage



39

NAND Gate

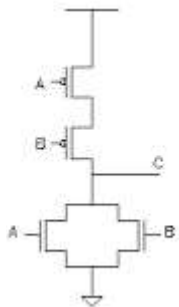
- 74HC00



40

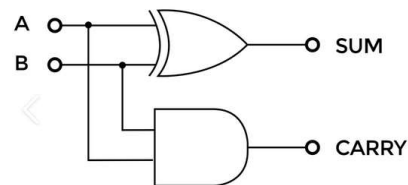
NOR Gate

- 74HC02

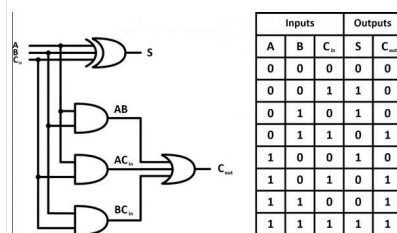


41

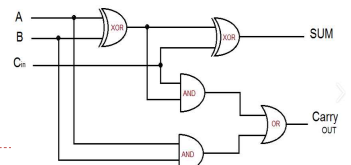
Combinational Logic Unit Example: Adder



A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

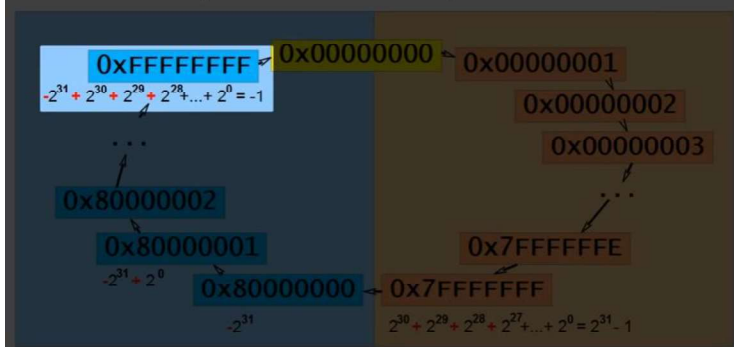


Full Adder



How about subtraction?

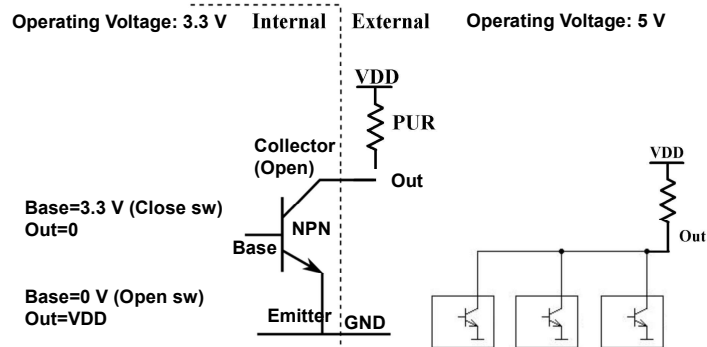
2's Complement Representation



43

Open Collector (OC) Gates

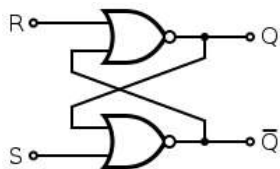
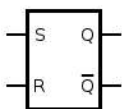
- Interface different operating voltage regions
- Connecting multiple OC outputs together in single line



44

Latches

- a device with exactly two stable states.
 - High-output and low-output.
 - Has a feedback path
 - information can be retained (stored)
 - One bit memory device
 - Can store one bit of data for as long as the device is powered



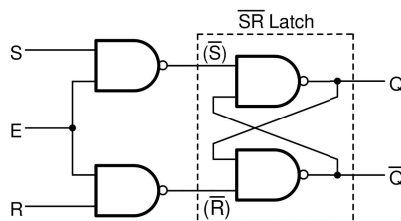
S	R	Q	Q̄
0	0	Latched	Latched
0	1	0	1
1	0	1	0
1	1	Metastable	Metastable

Keep Previous state

45

Gated SR Latch

- Enable/Disable functionality
- Can be done with NOR Gates as well

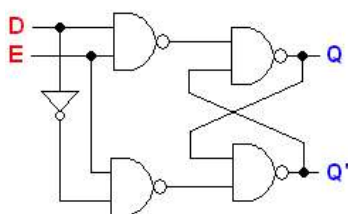


Enable	S	R	Q	Q̄
0	0	0	Latched	Latched
0	0	1	Latched	Latched
0	1	0	Latched	Latched
0	1	1	Latched	Latched
1	0	0	Latched	Latched
1	0	1	0	1
1	1	0	1	0
1	1	1	Metastable	Metastable

46

D Latch

- Remove illegal state

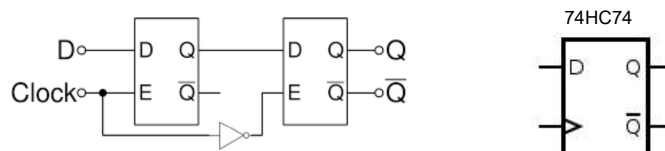


Enable	D	Q	Q̄
0	0	Latched	Latched
0	1	Latched	Latched
1	0	0	1
1	1	1	0

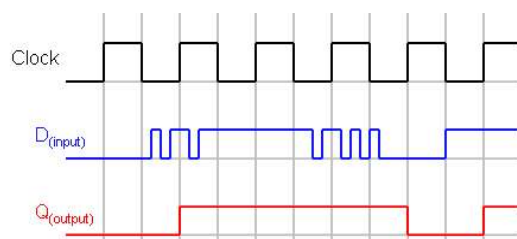
47

D FlipFlops (Registers)

- Uses two latches



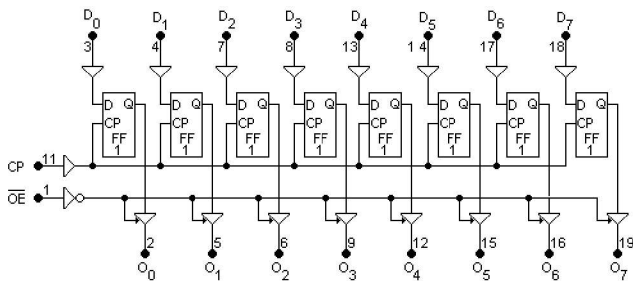
Negative or Positive FF?



48

Registers

- ▶ One-Bit FF or Multiple-Bit FFs
- ▶ 8 Bit FF or 8 bit register

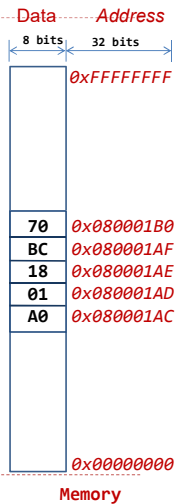


49

Memory

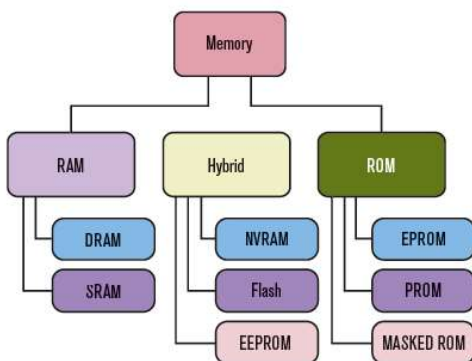
- ▶ Memory is arranged as a series of "locations"
 - ▶ Each location has a unique "address"
 - ▶ Each location holds a byte (**byte-addressable**)
 - ▶ e.g. the memory location at address 0x080001B0 contains the byte value 0x70, i.e., 112
- ▶ The number of locations in memory is limited
 - ▶ e.g. 4 GB of RAM
 - ▶ 1 Gigabyte (GB) = 2^{30} bytes
 - ▶ 2^{32} locations → 4,294,967,296 locations!
- ▶ Values stored at each location can represent either **program data** or **program instructions**.

50 e.g. the value 0x70 might be the code used to tell the processor to add two values together



Memory Types

- ▶ The memory used in embedded systems can be either internal or external.
 - ▶ The internal memory of a processor is very limited.



51

RAM (Random Access Memory)

- ▶ Can be accessed randomly
 - ▶ Memory location can be accessed without touching other locations
 - ▶ Can perform both read and write operations
- ▶ Static RAM :
 - ▶ Retains its contents as long as electrical power is being supplied to the device.
 - ▶ For Example: The desktop PC contains SRAM, which loses data when power is switched off.
- ▶ Dynamic RAM :
 - ▶ Needs refreshing
 - ▶ Typically one fourth of second, even if power is being supplied.
 - ▶ Uses DRAM controller in conjunction with DRAM to refresh the memory periodically
 - ▶ Less area.

52

ROM (Read Only Memory)

- ▶ Only read operation
 - ▶ cannot write data. Like CD.
- ▶ Store programs and data (permanently)
 - ▶ to initialize and operate embedded systems
- ▶ Retains its contents even if power is switched off;
- ▶ PROM(Programmable ROM)
 - ▶ Can be programmed only once.
- ▶ EPROM (Erasable ROM)
 - ▶ Can be programmed many times
 - ▶ An EPROM programmer is required to program the EPROM chip.

53

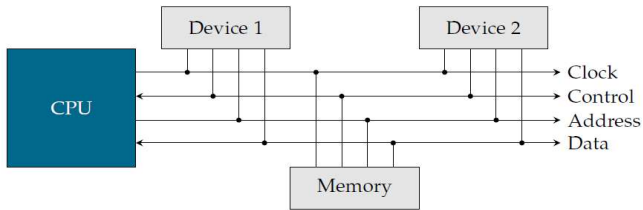
Hybrid Memory

- ▶ Works like both **RAM** and **ROM**.
 - ▶ Can write information (like RAM)
 - ▶ Information isn't erased when the power is off (like **ROM**)
- ▶ Electrically Erasable Programmable ROM (EEPROM)
 - ▶ Similar to EPROM
 - ▶ Erased electrically by applying an electrical signal to one of the pins.
- ▶ Non-Volatile RAM (NVRAM)
 - ▶ SRAM with battery backup
 - ▶ The contents are not erased even if power is switched off
 - ▶ Very expensive but data access is fast.
- ▶ Flash Memory
 - ▶ A Non - volatile memory, fast EEPROM, and electrically programmable
 - ▶ Can be erased one block at a time and programmed one bit at a time
 - ▶ Fast to read but slow to write
 - ▶ High density, low cost
 - ▶ Extensively used for embedded systems with embedded OS and the application program

54

The CPU bus

- ▶ CPU Bus is collection of wires, and the protocol, by which the CPU communicates with memory and



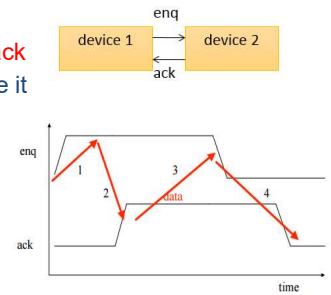
- ▶ The CPU is the bus master: it initiates all transfers
- ▶ Control: e.g. data ready, read/write

55

Bus Protocols

- ▶ Bus protocol determines how devices communicate
- ▶ The basic building block of most bus protocols is the **four-cycle handshake**

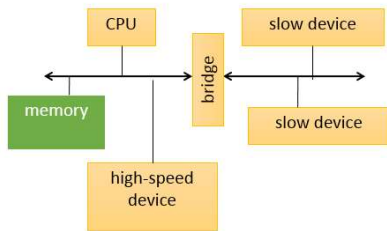
1. Device 1 raises **enq**
2. Device 2 responds with **ack**
3. Device 1 lowers **enq** once it has finished.
4. Device 2 lowers **ack**



56

System Bus Configurations

- ▶ A microprocessor system often has more than one bus
 - ▶ High-speed devices connected to a high-performance bus
 - ▶ Lower-speed devices are connected to a different bus
 - ▶ a **bridge** allows the buses to connect to each other

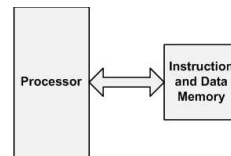


57

Computer Architecture

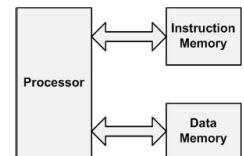
Von-Neumann

Instructions and data are stored in the same memory.



Harvard

Data and instructions are stored into separate memories.

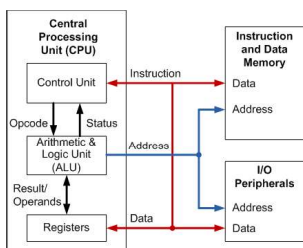


▶ 58

Computer Architecture

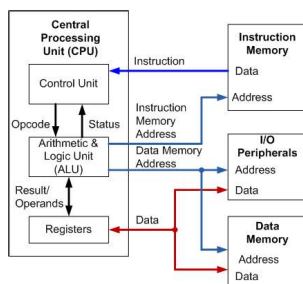
Von-Neumann

Instructions and data are stored in the same memory.



Harvard

Data and instructions are stored into separate memories.

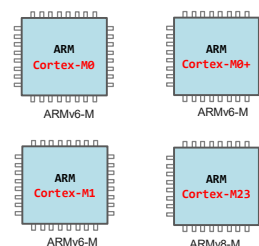


▶ 59

ARM Cortex-M Series Family

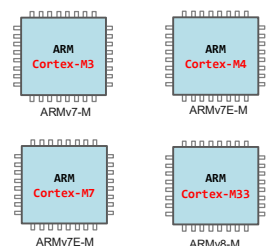
Von-Neumann

Instructions and data are stored in the same memory.



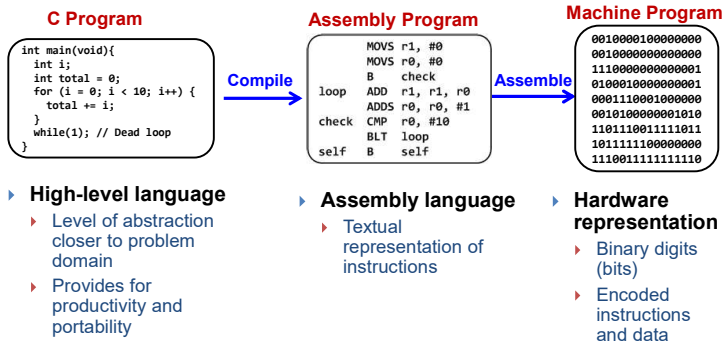
Harvard

Data and instructions are stored into separate memories.



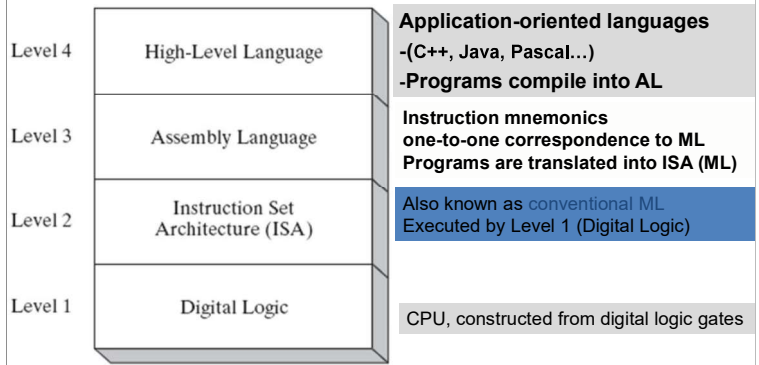
▶ 60

Levels of Program Code



61

Levels of Program Code



62

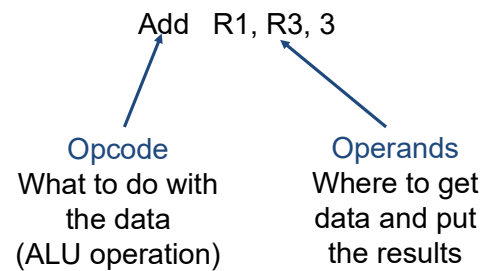
Assembly Languages

- One step up from machine language
- Originally a more user-friendly way to program
- Now mostly a compiler target
- Model of computation: stored program computer

63

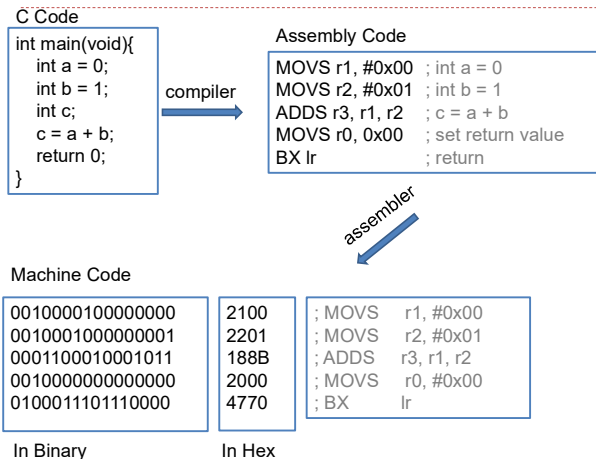
Assembly Language Instructions

- Built from two pieces



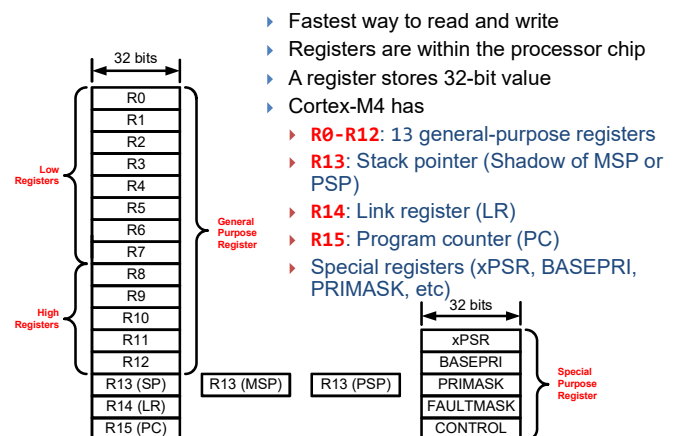
64

See a Program Runs



65

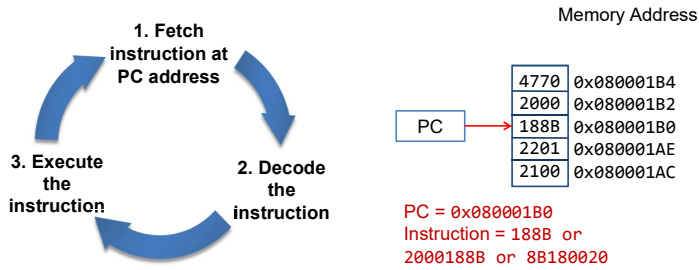
Processor Registers



66

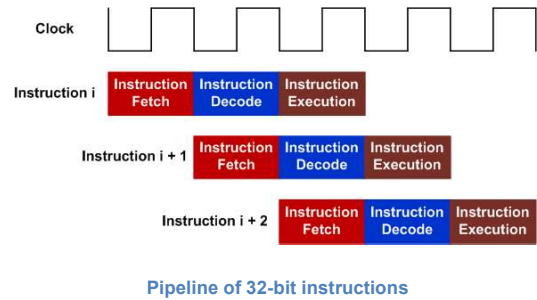
Program Execution

- **Program Counter (PC)** is a register that holds the memory address of the next instruction to be fetched from the memory.

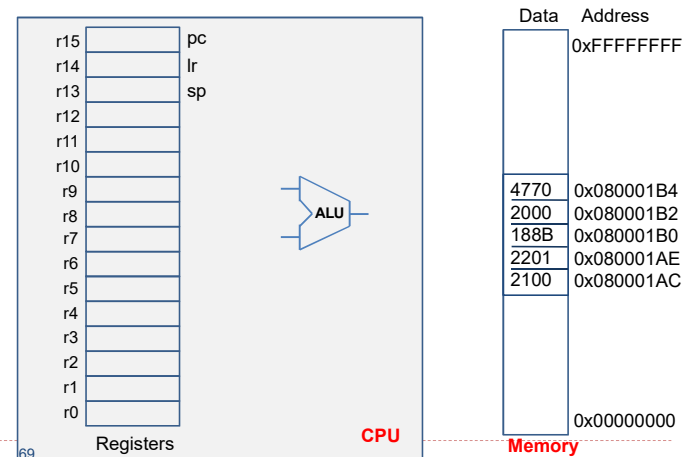


3-state pipeline: Fetch, Decode, Execution

- **Pipelining** allows hardware resources to be fully utilized
- One 32-bit instruction or two 16-bit instructions can be fetched.

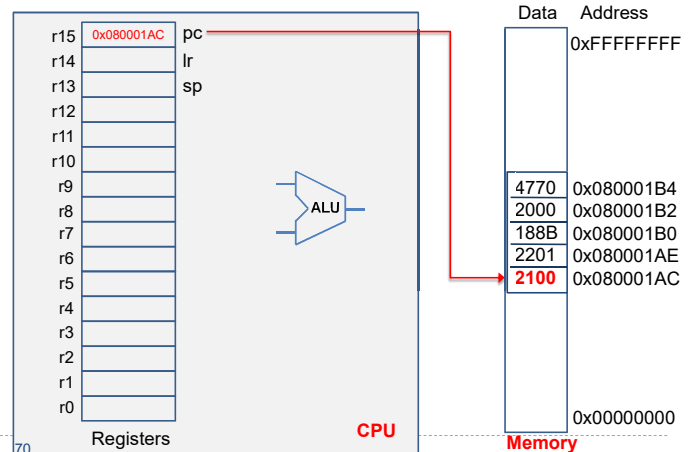


Machine codes are stored in memory



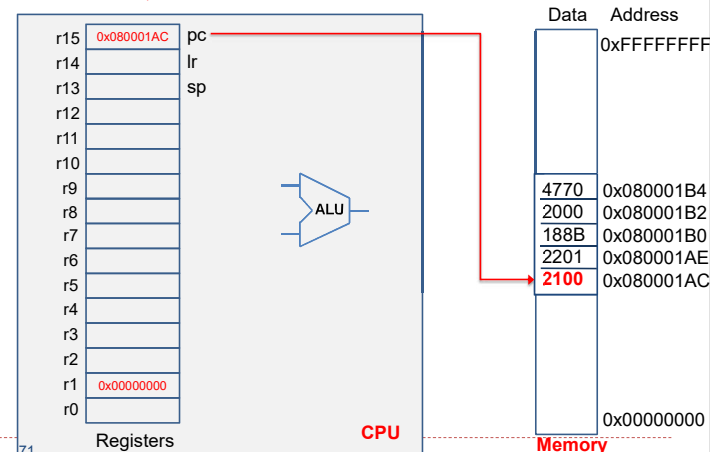
Fetch Instruction: pc = 0x08001AC

Decode Instruction: 2100 = MOV r1, #0x00



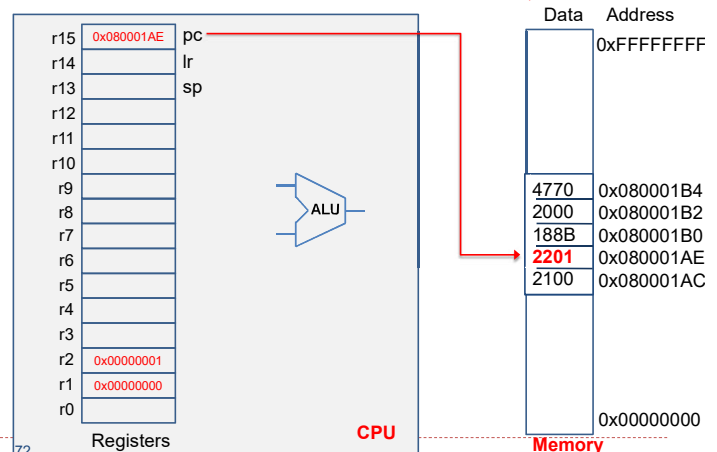
Execute Instruction:

MOV r1, #0x00

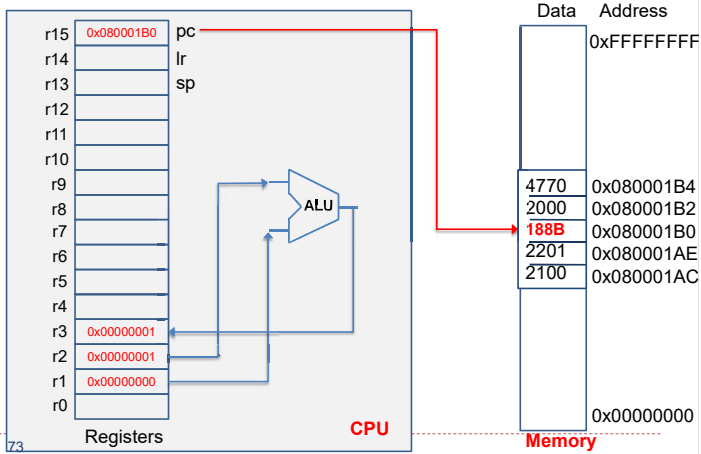


Fetch Next Instruction: pc = pc + 2

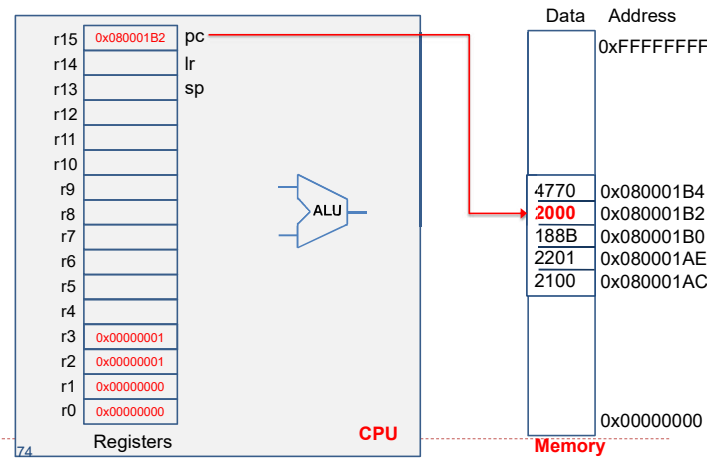
Decode & Execute: 2201 = MOV r2, #0x01



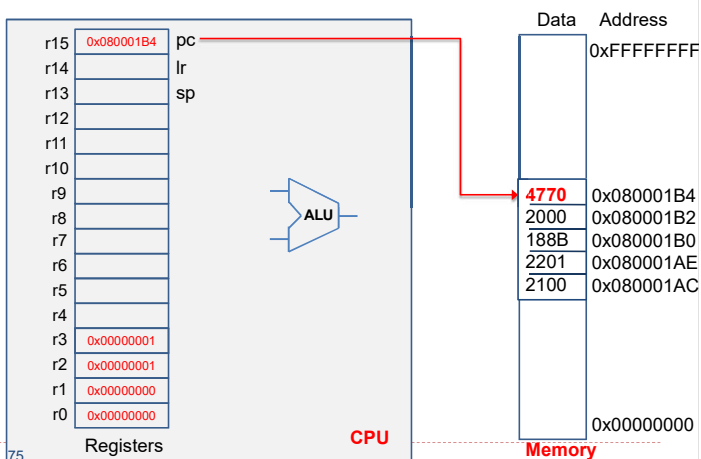
Fetch Next Instruction: $pc = pc + 2$
Decode & Execute: 188B = ADDS r3, r1, r2



Fetch Next Instruction: $pc = pc + 2$
Decode & Execute: 2000 = MOVS r0, #0x00



Fetch Next Instruction: $pc = pc + 2$
Decode & Decode: 4770 = BX lr



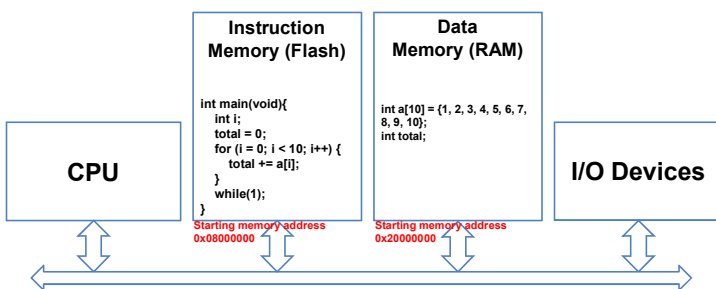
Example: Calculate the Sum of an Array

```
int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int total;
```

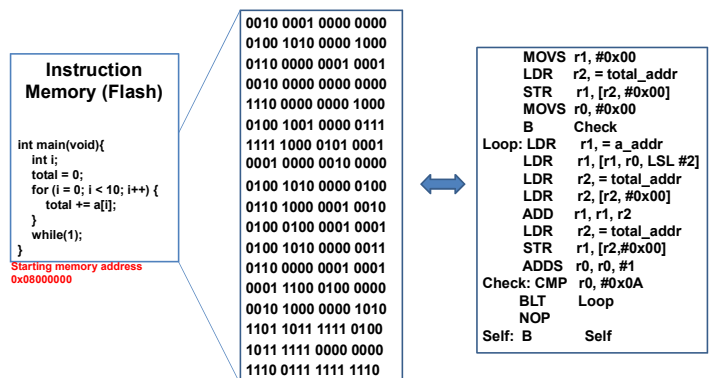
```
int main(void){
    int i;
    total = 0;
    for (i = 0; i < 10; i++) {
        total += a[i];
    }
    while(1);
}
```

76

Example: Calculate the Sum of an Array

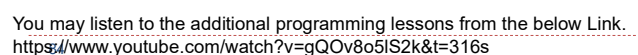


Example: Calculate the Sum of an Array



78

77



Doing something while counting:

```
int main() {  
    int counter = 0;  
    while (counter < 21) {  
        ++counter;  
        if ((counter & 1) != 0) {  
            /* do something when the counter is odd */  
        }  
    }  
    return 0;  
}
```



0001 0010	== 18	0001 0101	== 21
0000 0001	== 1	0000 0001	== 1
-----		-----	
0000 0000	== 0	0000 0001	== 1

```
int main() {  
    int counter = 0;  
    while (counter < 21) {  
        ++counter;  
        if ((counter & 1) != 0) {  
            /* do something when the counter is odd */  
        }  
        else {  
            /* do something when the counter is even */  
        }  
    }  
    return 0;  
}
```